

NP-Complejidad

Comentario: Hacer por lo menos los ejercicios 1 al 4. El ejercicio restante es un poco más difícil, pero de todos modos intentar resolverlo.

Ejercicio 1. Responder breve y claramente:

a. Dar un esquema de prueba de la transitividad de las reducciones polinomiales. Ayuda: composición de funciones de ciertas características computables eficientemente.

Si tengo lenguajes L_1 , L_2 y L_3 , y existe $L_1 \leq_p L_2$, y $L_2 \leq_p L_3 \rightarrow$ seguro $L_1 \leq_p L_3$

¿Por qué?

- Existe una f que es una función de reducción de L_1 a L_2 en tiempo polinomial
- Existe una g que es una función de reducción polinomial de L_2 a L_3

Entonces:

- Se nota fácilmente que podemos contruir una nueva función de reducción h que:
 - Primero llama a f
 - Luego llama a g
- Como las primeras dos, por definición de función de reducción, ya eran correctas y total computable en tiempo polinomial, la nueva función h claramente sigue siendo correcta, y $\text{poly}(n) + \text{poly}(n) = \text{poly}(n)$.

En criollo: Si el traductor de A a B es rápido, y el de B a C también, podés poner uno atrás del otro y vas a tener un traductor de A a C que sigue siendo rápido :)

b. ¿Cuándo un lenguaje es NP-difícil y cuándo es NP-completo?

Un lenguaje es **NP-Difícil** cuando todos los lenguajes de la clase NP se pueden reducir a él
(Es decir $L_{NP} \leq_p L_{NPH}$)

(es igual o más difícil que todos los lenguajes de NP)

Un lenguaje es NP-Completo, cuando cumple 2 condiciones:

- Pertenece a la clase NP
- Es NP-Difícil

Notar que un lenguaje NP-Difícil puede no ser de la clase NP, porque podría ser Exponencial o incluso estar fuera de R

c. ¿Por qué si $P \neq NP$, un lenguaje NP-completo no pertenece a P?

Por definición, un lenguaje L es NP-completo si es uno de los "más difíciles" de NP, lo que significa que cualquier otro problema en NP puede reducirse a él en tiempo polinomial ($NP \leq_p L$).

Si ese lenguaje L perteneciera a P, podrías usar esa reducción para resolver cualquier problema de NP en tiempo polinomial, lo que demostraría la igualdad $P=NP$, contradiciendo la premisa del enunciado.

d. Enunciar el esquema visto en clase para agregar un lenguaje a la clase NPC.

- **Primero demostrar la primera condición:**
Tenés que probar que $L_1 \in NP$. Es decir, que si el mago Merlin te tira una pista (certificado), podés chequear si es válida en un tiempo razonable (polinomial).
- **Segundo, que le gane al molino de viento (? y sea NP-Difícil:**
Agarrar un lenguaje que ya sabemos que está en NP-Completo (por ejemplo, SAT o 3-SAT) y demostrar que ese lenguaje se puede reducir a L_1
($SAT \leq_p L_1$)
- **Entonces, sabemos que:**
 - Si todo el mundo en NP se reduce al SAT (el NPC consagrado)
 - Y demostramos que ese NPC se reduce al nuestro L_1 .
 - Entonces, por transitividad, **todos los NP del universo se reducen a L_1 .**
 - ERGO, L_1 es NP-Completo

e. ¿Cuándo se sospecha que un lenguaje de NP está en NPI?

- **1) Requisito $P \neq NP$**
Si resultara que $P = NP$, la clase NPI directamente desaparece (sería un conjunto vacío). Solo hablamos de problemas intermedios bajo la sospecha de que las dos clases grandes, P y NP, son distintas
- **2) No se encuentra algoritmo en P:**
Si no podemos encontrar una MT M que resuelva el problema en tiempo polinomial, el lenguaje no estaría en P $\rightarrow$ sería NP
- **3) No se logra demostrar que sea NP-Completo:**
Si podemos encontrar una reducción desde un problema como SAT hacia él $\rightarrow$ no podemos demostrar que es "tan difícil como el más difícil".

Si se cumplen (1), (2) y (3) sospechamos entonces que el lenguaje está en NP-I

Ejercicio 2. Probar:

a. Si $L_1 \in \text{NPC}$ y $L_2 \in \text{NPC}$, entonces $L_1 \leq_p L_2$ y $L_2 \leq_p L_1$.

- El enunciado nos dice que L_1 y L_2 son NP-Completo, por tanto:
 - Ambos pertenecen a NP
 - Ambos son NPH: todos los lenguajes de NP se pueden reducir a ambos
- Prueba 1:
 - Dado que $L_1 \in \text{NP}$ y que L_2 es NP-H
 - Por definición de NP-H, todos los NP se pueden reducir a L_2
 - Y L_1 es de esos NP, ergo se puede reducir polinomialmente L_1 a L_2 !
- Prueba 2: Exactamente lo mismo pero invirtiendo los roles
 - Dado que $L_2 \in \text{NP}$ y que L_1 es NP-H
 - Por definición de NP-H, todos los NP se pueden reducir a L_1
 - Y L_2 es de esos NP, ergo se puede reducir polinomialmente L_2 a L_1 !

b. Si $L1 \leq_P L2$, $L2 \leq_P L1$, y $L1 \in NPC$, entonces $L2 \in NPC$.

Ayuda: recurrir directamente a la definición de NPC.

Para probar que $L2 \in NPC$, tenemos que recurrir al esquema que vimos en el 1d.

- 1) Demostrar que está en NP:

Partimos que $L1 \in NPC$, **por definición de NPC**, $L1 \in NP$.

Y el enunciado nos dice que $L2 \leq L1$.

Lo que significa que existe una función de reducción que convierte instancias de $L2$, en $L1$ en tiempo polinomial

Como existe una función de reducción con la que podemos transformar $L2$ en un problema que ya sabemos que es NP, entonces $L2$ también pertenece a NP.

- 2) Demostrar que es NP-Difícil:

Para que un lenguaje sea NP-Completo, **todos** los lenguajes de NP deben poder reducirse a él. Digamos un language L_{NP}

Partimos que $L1 \in NPC$, por definición de NPC $L1$ es NPH y todos los lenguajes de NP se pueden reducir a él.

Y el enunciado nos dice que $L1 \leq_P L2$.

Entonces, por transitividad:

- $L_{NP} \leq_P L1 \leq_P L2 \rightarrow \text{Cualquier } L_{NP} \leq_P L2 \rightarrow \text{Cumple ser NP-H}$

Como probamos que $L2 \in NP$ y $L2$ es NP-H, entonces queda probado que **$L2 \in NPC$**

Ejercicio 3. Un lenguaje es CO-NP-completo sii todos los lenguajes de CO-NP se reducen polinomialmente a él. Probar que SAT^c es CO-NP-completo.

Ayuda: $L1 \leq L2 \leftrightarrow L1^c \leq L2^c$.

Contexto:

- **SAT^c :** SAT^c es el conjunto de todas las fórmulas booleanas que no son satisfacibles. Una fórmula pertenece a SAT^c si para todas las combinaciones posibles de valores de sus variables, el resultado de la fórmula es siempre falso.
- **CO-NP:** La clase CO-NP es el conjunto de lenguajes cuyos complementos pertenecen a la clase NP. Formalmente, un lenguaje L está en CO-NP si y solo si su complemento L^c está en NP.

Prueba:

Para probar que un lenguaje es CO-NP-completo, debemos demostrar dos condiciones: que pertenece a la clase CO-NP y que todos los lenguajes de la clase se reducen a él

1) Pertenencia a CO-NP:

Esta primera condición es trivial:

- La clase CO-NP es el conjunto de lenguajes cuyos complementos pertenecen a la clase NP
- El complemento de SAT^c es SAT, y sabemos que SAT pertenece a NP, ergo, SAT^c está en CO-NP.

2) Probar que todos los CO-NP se reducen a SAT^c :

Sea L' un lenguaje cualquiera perteneciente a CO-NP

Por la definición de la clase, esto implica que su complemento L'^c pertenece a NP

Sabemos que SAT es NP-Completo, lo que significa que todos los NP se reducen a él. Incluido el L'^c que acabo de definir.

Por lo tanto, existe una reducción $L'^c \leq SAT$

Usando la propiedad: $L1 \leq L2 \leftrightarrow (L1)^c \leq (L2)^c$

Obtenemos que: $L'^c \leq SAT \leftrightarrow (L'^c)^c \leq (SAT)^c$

Y simplificando: $L'^c \leq SAT \leftrightarrow L' \leq (SAT)^c$

Como SAT^c pertenece a CO-NP y cualquier lenguaje L' de esa clase se puede reducir polinomialmente a él, queda probado que SAT^c es CO-NP-completo.

Ejercicio 4. Sean los lenguajes A y B, tales que $A \neq \emptyset$, $A \neq \Sigma^*$, y $B \in P$.

Probar: $(A \cap B) \leq_p A$.

Ayuda: intentar con una reducción polinomial que, dada una cadena w, lo primero que haga sea chequear si $w \in B$, teniendo en cuenta que existe un elemento e que no está en A.

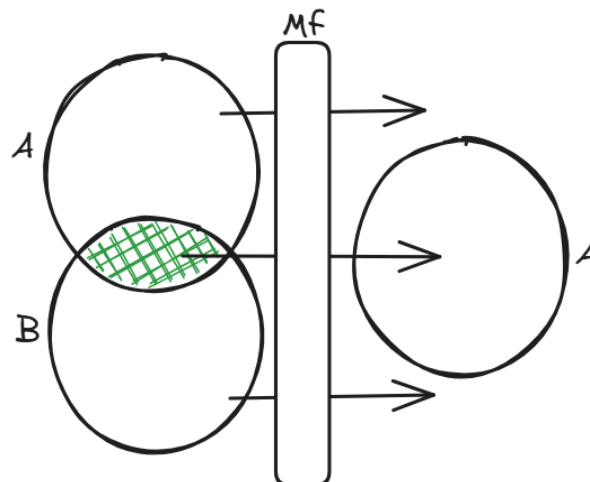
- Qué significa $A \neq \emptyset$ y $A \neq \Sigma^*$?

El lenguaje A No es la nada misma ($\emptyset$) y tampoco es el universo entero (Σ^*).
Es decir:

- Existe al menos una cadena $b \in A$.
- Existe al menos una cadena $b \notin A$ (un elemento de error)

- **Reducción:**

Queremos que nuestra función $f(w)$ devuelva algo que esté en A solo si w cumple las dos condiciones (estar en A y estar en B).



- **Idea general de f:**

- **Si $w \in B$:** El problema se reduce a saber si $w \in A$. Entonces, le pasamos el mismo w sin modificar a la máquina que decide A.
- **Si $w \notin B$:** Ya sabemos que w no puede estar en la intersección $(A \cap B)$. Entonces, nuestra función tiene que devolver algo (elemento e) que no esté en A sí o sí, para que la equivalencia sea falsa de los dos lados. (elemento que seguro existe, porque nos dicen que $A \neq \Sigma^*$)

- **MTf es total computable en tiempo $\text{poly}(n)$**

- Chequear si $w \in B$ toma tiempo polinomial porque nos aseguran $B \in P$
- Pasar el mismo w o el elemento e en la cinta de salida toma tiempo lineal respecto a la entrada.
- $\text{poly}(n) + n = O(\text{poly})$

- **Correctitud de MT f:**

- $w \in A \cap B \rightarrow w \in A \wedge w \in B \rightarrow f(w) = w \rightarrow f(w) \in A$
Si w está en la intersección, por definición está en B , por lo que la función devuelve w . Como también estaba en A , el resultado pertenece a A .
- $w \notin A \cap B \rightarrow w \notin B \vee (w \notin A \wedge w \in B) \rightarrow (f(w) = e \vee f(w) = w) \text{ con } f(w) \notin A \rightarrow f(w) \notin A$
Si w no está en la intersección, puede ser porque no está en B (la función devuelve el $e \notin A$) o porque estando en B no pertenece a A (la función devuelve $w \notin A$). En ambos casos, el resultado queda fuera de A .

Ejercicio 5. Sea el lenguaje $SH-s-t = \{(G, s, t) \mid G \text{ es un grafo no dirigido y tiene un camino de Hamilton del vértice } s \text{ al vértice } t\}$. Un grafo $G = (V, E)$ tiene un camino de Hamilton del vértice s al vértice t sii G tiene un camino entre s y t que recorre todos los vértices restantes una sola vez. Probar que $SH-s-t$ es NP-completo. Ayuda: se sabe que CH, el lenguaje correspondiente al problema del circuito hamiltoniano, es NP-completo.

Ni idea, si me pongo a hacer este ejercicio no llego al parcial :(

